PolicyPal — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── data/
│   ├── policies/
│   │   ├── leave_policy.md
│   │   └── remote_policy.md
│   ├── pause_demo.py
│   └── resume_demo.py
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── approval_agent.py
│   │   ├── reasoner_agent.py
│   │   └── retriever_agent.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── prompts/
│   │   ├── reasoner.txt
│   │   └── router.txt
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── chunking.py
│   │   ├── ingest.py
│   │   ├── retriever.py
│   │   └── store.py
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py
│   │   └── prompts.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── graph.py
```

```
│   └── state.py
├── tests/
│   ├── integration/
│   │   └── test_live_graph.py
│   └── unit/
│       ├── test_approval_checks.py
│       ├── test_chunking.py
│       ├── test_config.py
│       ├── test_retrieval.py
│       └── test_routing.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
└── requirements.txt
```

Setup files

`requirements.txt`

```
langchain-core>=0.3
langchain-openai>=0.2
langgraph>=0.2
langgraph-checkpoint-sqlite>=2.0
chromadb>=0.5
pydantic>=2.0
python-dotenv>=1.0
```

`.env.example`

```
OPENAI_API_KEY=sk-your-key-here
LOG_LEVEL=INFO
```

`.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
chroma_db/
policypal_state.sqlite
```

Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

The policy documents

```
# data/policies/leave_policy.md
Paid leave: full-time employees receive 20 paid leave days per calendar
year. Unused days carry over up to a maximum of 5 days. Leave must be
requested at least 3 working days in advance.
```

```
# data/policies/remote_policy.md
Remote work: employees may work remotely up to two days per week with
manager approval. Core collaboration hours are 10:00-15:00 local time.
```

Settings, errors, logging, prompts

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
```

```
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/utils/prompts.py

def load_prompt(name):
    with open("src/prompts" + "/" + name + ".txt") as prompt_file:
        return prompt_file.read().strip()
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The RAG layer

```
# src/rag/chunking.py
# Split a document into paragraph chunks (Doc08). Small docs stay whole.
def chunk_by_paragraph(text):
    chunks = []
    for block in text.split("\n\n"):
        cleaned = block.strip()
```

```
        if cleaned != "":
            chunks.append(cleaned)
    return chunks
```

```
# src/rag/store.py
# One place that builds the Chroma collection. The embedding function is a
# parameter so tests can pass a fast, offline one; the app uses Chroma's
# default. Distance is squared-L2: LOWER means closer.
import chromadb

COLLECTION_NAME = "hr_policies"

def get_collection(embedding_function=None):
    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    if embedding_function is None:
        return client.create_collection(COLLECTION_NAME)
    return client.create_collection(
        COLLECTION_NAME, embedding_function=embedding_function
    )
```

```
# src/rag/ingest.py
# Read every policy file, chunk it, and add the chunks to the collection
# with their source file in the metadata — so an answer can be cited.
import os

from src.rag.chunking import chunk_by_paragraph
from src.utils.logger import get_logger

def build_vector_store(collection, policies_dir="data/policies"):
    logger = get_logger(__name__)
    ids = []
    documents = []
    metadatas = []
    counter = 0
    for name in sorted(os.listdir(policies_dir)):
        if not name.endswith(".md"):
            continue
        with open(os.path.join(policies_dir, name)) as policy_file:
            text = policy_file.read()
        for chunk in chunk_by_paragraph(text):
            ids.append(f"chunk-{counter}")
```

```
            documents.append(chunk)
            metadatas.append({"source": name})
            counter = counter + 1
    collection.add(ids=ids, documents=documents, metadatas=metadatas)
    logger.info(f"ingested {len(ids)} chunks")
    return collection
```

```
# src/rag/retriever.py
# retrieve(): the k closest chunks as dicts with their source and distance.
def retrieve(collection, query, k=3):
    found = collection.query(query_texts=[query], n_results=k)
    passages = []
    for index in range(len(found["documents"][0])):
        passages.append({
            "text": found["documents"][0][index],
            "source": found["metadatas"][0][index]["source"],
            "distance": found["distances"][0][index],
        })
    return passages
```

The state and the three roles

```
# src/state.py
from typing import TypedDict

class DocState(TypedDict):
    question: str
    kind: str          # "question" | "action"
    passages: list     # retrieved chunks (may be empty = not found)
    answer: str
    approved: bool
    result: str

def start_state(question):
    return {
        "question": question,
        "kind": "",
        "passages": [],
        "answer": "",
        "approved": False,
        "result": "",
    }
```

```
# src/prompts/router.txt
(Routing is done in code, not by the model — see classify_kind. This file
is kept only so the prompts folder documents every role.)
```

```
# src/agents/retriever_agent.py
# The retriever role: route question-vs-action (in CODE, so it's testable
# with no model), then search and keep only passages close enough to be
# relevant. An empty result is the "not found" signal.
from src.rag.retriever import retrieve
from src.utils.logger import get_logger

MAX_DISTANCE = 1.2   # squared-L2; above this a chunk isn't really relevant

def classify_kind(question):
    lowered = question.lower()
    for word in ["file", "submit", "request", "book", "cancel", "apply"]:
        if word in lowered:
            return "action"
    return "question"

def search(collection, question):
    logger = get_logger(__name__)
    passages = retrieve(collection, question)
    relevant = []
    for passage in passages:
        if passage["distance"] <= MAX_DISTANCE:
            relevant.append(passage)
    logger.info(f"retrieved {len(relevant)} relevant passages")
    return relevant
```

```
# src/prompts/reasoner.txt
You answer an HR question using ONLY the policy passages below. Quote the
relevant rule and cite its source file in parentheses, like (leave_policy.md).
If the passages do not contain the answer, say exactly: "That isn't covered
in the HR policies." Never use outside knowledge.

Question: {question}

Policy passages:
{passages}
```

```
# src/agents/reasoner_agent.py
# The reasoner role: a grounded, cited answer from the retrieved passages
# only. With no passages it returns the honest "not covered" line without
```

```
# calling the model at all.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def answer_grounded(model, question, passages):
    logger = get_logger(__name__)
    if len(passages) == 0:
        logger.info("no passages -> not found")
        return "That isn't covered in the HR policies."
    lines = []
    for passage in passages:
        lines.append(f"[{passage['source']}] {passage['text']}")
    prompt = ChatPromptTemplate.from_template(load_prompt("reasoner"))
    chain = prompt | model | StrOutputParser()
    return chain.invoke({"question": question, "passages": "\n".join(lines)})
```

```
# src/agents/approval_agent.py
# The approval role: build a human checklist of the action, then PAUSE the
# graph with interrupt() and wait. A separate run resumes with yes/no.

def build_checklist(question):
    return (
        "PolicyPal is about to take an action on your behalf:\n"
        f"  - request: {question}\n"
        "Approve? (yes/no)"
    )
```

The graph

```
# src/graph.py
# The LangGraph graph: retrieve -> (reason | approval). The approval node
# pauses with interrupt(); a checkpointer saves the state so a separate run
# can resume it.
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command, interrupt

from src.agents.approval_agent import build_checklist
from src.agents.reasoner_agent import answer_grounded
from src.agents.retriever_agent import classify_kind, search
from src.state import DocState
```

```
def build_graph(model, collection, checkpointer=None):
    def retrieve_node(state):
        kind = classify_kind(state["question"])
        passages = search(collection, state["question"])
        if kind == "action":
            return Command(goto="approval",
                           update={"kind": kind, "passages": passages})
        return Command(goto="reason",
                       update={"kind": kind, "passages": passages})

    def reason_node(state):
        answer = answer_grounded(model, state["question"], state["passages"])
        return Command(goto=END, update={"answer": answer})

    def approval_node(state):
        decision = interrupt({"checklist": build_checklist(state["question"])})
        if decision == "yes":
            return Command(goto=END, update={
                "approved": True,
                "result": f"Recorded: {state['question']}",
            })
        return Command(goto=END, update={
            "approved": False, "result": "Action cancelled.",
        })

    graph = StateGraph(DocState)
    graph.add_node("retrieve", retrieve_node)
    graph.add_node("reason", reason_node)
    graph.add_node("approval", approval_node)
    graph.add_edge(START, "retrieve")
    return graph.compile(checkpointer=checkpointer)
```

The entry point and the pause/resume demos

```
# main.py
import sys

from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from src.config import load_config
from src.exceptions import MissingConfigError
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.ingest import build_vector_store
```

```
from src.rag.store import get_collection
from src.state import start_state

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    model = get_model(config)
    collection = build_vector_store(get_collection())
    app = build_graph(model, collection, checkpointer=MemorySaver())
    print("PolicyPal — ask a policy question or request an action, /quit.")
    thread = 0
    while True:
        question = input("You: ").strip()
        if question == "/quit":
            break
        if question == "":
            continue
        thread = thread + 1
        config_run = {"configurable": {"thread_id": str(thread)}}
        result = app.invoke(start_state(question), config_run)
        if app.get_state(config_run).next:
            print(result["__interrupt__"][0].value["checklist"])
            decision = input("Approve (yes/no): ").strip()
            result = app.invoke(Command(resume=decision), config_run)
            print(result["result"])
        else:
            print(f"{result['answer']}")

if __name__ == "__main__":
    main()
```

```
# data/pause_demo.py
# Part 1: start an action run and stop at the pause. The state is SAVED to
# a SQLite file, so resume_demo.py (a separate process) can finish it.
# Run from the repo root: python -m data.pause_demo
from langgraph.checkpoint.sqlite import SqliteSaver

from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.ingest import build_vector_store
from src.rag.store import get_collection
```

```
from src.state import start_state

DB_PATH = "policypal_state.sqlite"
THREAD = {"configurable": {"thread_id": "demo"}}

def main():
    config = load_config()
    model = get_model(config)
    collection = build_vector_store(get_collection())
    with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
        app = build_graph(model, collection, checkpointer=checkpointer)
        app.invoke(start_state("File a leave request for Monday"), THREAD)
        paused = app.get_state(THREAD)
        print("Paused at:", paused.next)
        print(paused.values.get("question"))

if __name__ == "__main__":
    main()
```

```
# data/resume_demo.py
# Part 2: a NEW process opens the same saved state and approves it.
# Run from the repo root: python -m data.resume_demo
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import Command

from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.ingest import build_vector_store
from src.rag.store import get_collection

DB_PATH = "policypal_state.sqlite"
THREAD = {"configurable": {"thread_id": "demo"}}

def main():
    config = load_config()
    model = get_model(config)
    collection = build_vector_store(get_collection())
    with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
        app = build_graph(model, collection, checkpointer=checkpointer)
        final = app.invoke(Command(resume="yes"), THREAD)
        print("Resumed:", final["result"])

if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "POLICYPAL_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_chunking.py
# Run from the repo root: python -m tests.unit.test_chunking
from src.rag.chunking import chunk_by_paragraph
```

```
def test_splits_paragraphs():
    chunks = chunk_by_paragraph("one\n\ntwo\n\nthree")
    result = "OK"
    if chunks != ["one", "two", "three"]:
        result = "WRONG"
    print(f"case 1 (splits on blank lines) -> {result}")

def test_drops_empty():
    chunks = chunk_by_paragraph("one\n\n\n\n   \n\ntwo")
    result = "OK"
    if chunks != ["one", "two"]:
        result = "WRONG"
    print(f"case 2 (drops empty blocks)    -> {result}")

if __name__ == "__main__":
    test_splits_paragraphs()
    test_drops_empty()
```

```
# tests/unit/test_routing.py
# Routing is code, not a model call. Run from the repo root:
#   python -m tests.unit.test_routing
from src.agents.retriever_agent import classify_kind

def test_question_routes_to_question():
    result = "OK"
    if classify_kind("How many paid leave days do I get?") != "question":
        result = "WRONG"
    print(f"case 1 (plain question) -> {result}")

def test_action_routes_to_action():
    result = "OK"
    if classify_kind("File a leave request for Monday") != "action":
        result = "WRONG"
    print(f"case 2 (action request) -> {result}")

if __name__ == "__main__":
    test_question_routes_to_question()
    test_action_routes_to_action()
```

```
# tests/unit/test_retrieval.py
# Uses a fast, offline keyword embedding so there's no model download and
# no network. The vectors are L2-normalized, so the distances fall in the
# same 0-2 range as Chroma's default embeddings — which is what MAX_DISTANCE
# (1.2) is calibrated for. Run from the repo root:
```

```
#   python -m tests.unit.test_retrieval
import math

from chromadb.utils.embedding_functions import EmbeddingFunction

from src.agents.retriever_agent import search
from src.rag.ingest import build_vector_store
from src.rag.store import get_collection

VOCAB = ["leave", "paid", "days", "remote", "work", "hours", "wifi",
         "password", "berlin"]

class KeywordEmbedding(EmbeddingFunction):
    def __call__(self, texts):
        vectors = []
        for text in texts:
            lowered = text.lower()
            counts = [float(lowered.count(word)) for word in VOCAB]
            norm = math.sqrt(sum(value * value for value in counts))
            if norm == 0:
                norm = 1.0
            vectors.append([value / norm for value in counts])
        return vectors

def build():
    collection = get_collection(embedding_function=KeywordEmbedding())
    return build_vector_store(collection)

def test_relevant_question_retrieves():
    passages = search(build(), "how many paid leave days")
    result = "OK"
    if len(passages) == 0:
        result = "WRONG"
    elif passages[0]["source"] != "leave_policy.md":
        result = "WRONG"
    print(f"case 1 (leave question -> leave_policy.md) -> {result}")

def test_offtopic_is_not_found():
    passages = search(build(), "wifi password berlin")
    result = "OK"
    if len(passages) != 0:
        result = "WRONG"
    print(f"case 2 (off-topic -> nothing relevant)     -> {result}")

if __name__ == "__main__":
    test_relevant_question_retrieves()
    test_offtopic_is_not_found()
```

```
# tests/unit/test_approval_checks.py
# Run from the repo root: python -m tests.unit.test_approval_checks
from src.agents.approval_agent import build_checklist

def test_checklist_names_the_action():
    text = build_checklist("File a leave request for Monday")
    result = "OK"
    if "File a leave request for Monday" not in text:
        result = "WRONG"
    if "Approve?" not in text:
        result = "WRONG"
    print(f"case 1 (checklist shows the action) -> {result}")

if __name__ == "__main__":
    test_checklist_names_the_action()
```

```
# tests/integration/test_live_graph.py
# The whole graph against the real model. Run from the repo root:
#   python -m tests.integration.test_live_graph
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.ingest import build_vector_store
from src.rag.store import get_collection
from src.state import start_state

def build_app():
    config = load_config()
    model = get_model(config)
    collection = build_vector_store(get_collection())
    return build_graph(model, collection, checkpointer=MemorySaver())

def run(app, question, thread):
    cfg = {"configurable": {"thread_id": thread}}
    result = app.invoke(start_state(question), cfg)
    return app, cfg, result

def test_grounded_answer_cites_source():
    app, cfg, result = run(build_app(), "How many paid leave days?", "g")
    ok = "20" in result["answer"] and "leave_policy.md" in result["answer"]
    print(f"case 1 (grounded + cited)   -> {'OK' if ok else 'WRONG'}")

def test_not_found():
```

```
    app, cfg, result = run(build_app(), "What's the Berlin wifi password?", "n")
    ok = "isn't covered" in result["answer"]
    print(f"case 2 (not found)          -> {'OK' if ok else 'WRONG'}")

def test_action_approved():
    app = build_app()
    cfg = {"configurable": {"thread_id": "a"}}
    app.invoke(start_state("File a leave request for Monday"), cfg)
    final = app.invoke(Command(resume="yes"), cfg)
    ok = final["approved"] and "Recorded" in final["result"]
    print(f"case 3 (action approved)    -> {'OK' if ok else 'WRONG'}")

def test_action_rejected():
    app = build_app()
    cfg = {"configurable": {"thread_id": "r"}}
    app.invoke(start_state("File a leave request for Monday"), cfg)
    final = app.invoke(Command(resume="no"), cfg)
    ok = (not final["approved"]) and "cancelled" in final["result"]
    print(f"case 4 (action rejected)    -> {'OK' if ok else 'WRONG'}")

if __name__ == "__main__":
    test_grounded_answer_cites_source()
    test_not_found()
    test_action_approved()
    test_action_rejected()
```

Expected unit output (all `OK`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (splits on blank lines) -> OK
case 2 (drops empty blocks)    -> OK
```

```
case 1 (plain question) -> OK
case 2 (action request) -> OK
```

```
case 1 (leave question -> leave_policy.md) -> OK
case 2 (off-topic -> nothing relevant)     -> OK
```

```
case 1 (checklist shows the action) -> OK
```

That's the whole project. If PolicyPal retrieves before it answers, cites the policy file, says "not covered" when the docs don't have it, and pauses for a human *yes* before any action — resumable from a separate run — you've built it.